



UNIVERSIDAD DE SANTIAGO DE CHILE

ESTRUCTURA DE COMPUTADORES Y SISTEMAS DISTRIBUIDOS · EVALUACIÓN 3 · CASO 2

Línea de producción con balanceo dinámico de carga

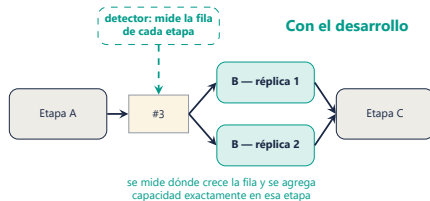
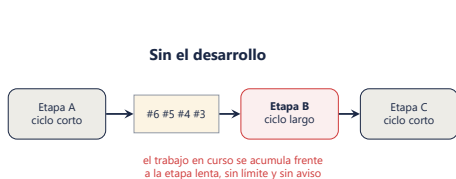
Sistema distribuido que simula una planta conservera, detecta su cuello de botella en vivo y demuestra qué pasa cuando se escala la etapa lenta

César Olivares · Simón García-Huidobro

Profesor: Daniel Calderón R. · 26 de agosto de 2026

EL PROBLEMA

Toda línea en serie produce al ritmo de su etapa más lenta — y sin medición no se sabe cuál es



- Dos problemas de ingeniería: **(1)** repartir el trabajo entre copias de la etapa lenta sin duplicar ni perder unidades, y **(2)** determinar, midiendo, **cuál etapa limita la producción**.
- Replicar sin medir es invertir a ciegas; medir sin poder replicar es diagnosticar sin tratamiento. Este desarrollo hace ambas cosas y las **verifica con experimentos**.

EL CASO

Esa lógica, aplicada a una planta conservera ficticia de jurel en lata: cuatro etapas en serie



- La unidad de trabajo es el **lote** (latas del mismo formato); una **estación** ejecuta una etapa; entre etapas hay una **cola**: la fila de lotes que esperan.
- Si a una etapa le llega trabajo más rápido de lo que procesa, su fila crece: eso es un **cuello de botella**.
- **Envasado** (ciclo 12 s) es la etapa más lenta: es la que se replica.

La pregunta del caso

¿Qué pasa con el cuello de botella cuando escalas la etapa lenta?

Adelanto: no desaparece — se muda.

TECNOLOGÍAS Y SU ROL

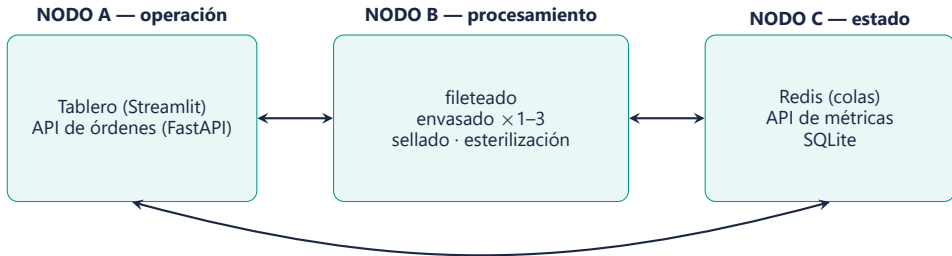
Qué es cada pieza y qué papel cumple en el sistema — los conceptos que usaremos el resto de la charla

Pieza	Qué es	Rol en este sistema
Python	Lenguaje de programación	Todos los servicios están escritos en él
Docker	Empaqueta un programa con su entorno en una imagen ; cada copia en ejecución es un contenedor	Cada servicio corre en su contenedor; replicar una etapa = lanzar más contenedores de la misma imagen
Redis	Almacén de datos en memoria; atiende comandos de a uno	Las colas de trabajo entre etapas; su comando BRPOP reparte los lotes (lo veremos)
FastAPI	Biblioteca de Python para construir APIs : interfaces HTTP por las que un programa recibe peticiones de otro	La API de órdenes (crear y consultar) y la API de métricas (espera, lead time, diagnóstico de cuello)
SQLite	Base de datos relacional contenida en un archivo	Registro persistente de órdenes y eventos de la línea
Streamlit	Biblioteca de Python para interfaces web	El tablero del operador

Cada comando o sigla que aparezca de aquí en adelante viene de esta tabla.

ARQUITECTURA EN NODOS

Un nodo lógico es un grupo de contenedores que correría en una máquina propia; hay tres, y cada uno escala por un motivo distinto



- Docker deja que los tres nodos convivan hoy en una máquina y mañana se separen, sin cambiar código: la frontera entre nodos ya es la frontera entre contenedores.
- **B** se replica bajo carga; **C** concentra el estado compartido; **A** escala con los usuarios. Separar nodos separa dominios de falla: si las estaciones saturan la CPU, el operador sigue atendido.

UNA IMAGEN, CUATRO ESTACIONES

El mismo programa, desplegado cuatro veces con configuración distinta — y por qué eso hace posible replicar

- Las cuatro estaciones son **una sola imagen de Docker** (un solo programa): al arrancar, cada contenedor recibe por variables de entorno *qué etapa es, de qué cola lee, a cuál escribe y cuánto dura su ciclo*.
- **¿Por qué replicar?** Envasado procesa un lote cada 12 s, menos de lo que le llega. La única forma de subir su capacidad es poner más envasadoras en paralelo.
- Con Docker, eso es **lanzar más contenedores de la misma imagen**: un comando, sin tocar configuración. Ninguna réplica sabe cuántas hermanas tiene.

El comando concreto — «levanta la línea con tres envasadoras»:

```
docker compose up -d  
--scale envasado=3
```

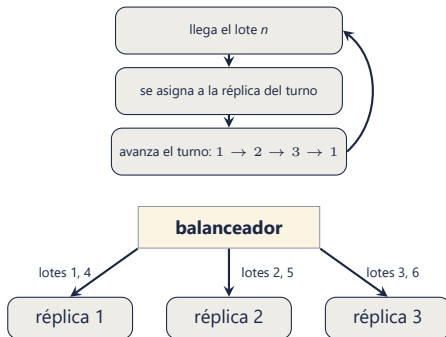
1 imagen

4 etapas · hasta 6 contenedores de estación

REPARTO POR TURNOS: ROUND-ROBIN

La forma clásica de balancear — y el punto de partida contra el que se compara nuestro desarrollo

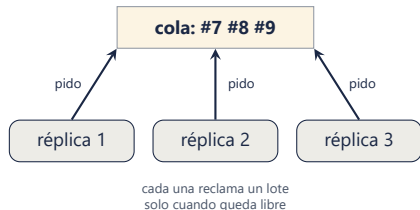
- *Round-robin* = el balanceador asigna el trabajo **por turnos fijos**: el lote 1 a la réplica 1, el 2 a la 2, el 3 a la 3, el 4 de vuelta a la 1...
- Es la opción por defecto de los balanceadores clásicos: simple y pareja **en cantidad**.
- Su límite: es **ciego al estado** de cada réplica. Si una está ocupada, lenta o caída, *igual le toca su turno* — y el trabajo se apila frente a ella.



BALANCEO POR DEMANDA

Nuestro desarrollo: las réplicas piden trabajo al quedar libres; nadie se los asigna

- Lo esperable era un balanceador que *asigna*, como el round-robin recién visto. Optamos por lo contrario —y lo defendemos—: **las réplicas piden**, y el balanceador **es la cola compartida**.
- Una réplica lenta simplemente **pide menos veces**: el reparto se adapta solo a la capacidad real. Eso es balanceo *dinámico*.



Verificado midiendo: con una réplica al doble de ciclo, el reparto fue **11/12/7** (experimento 4).

	Por turnos	Por demanda
Quién decide	el balanceador	la réplica libre
Réplica lenta	recibe igual	pide menos
Agregar réplicas	reconfigurar	basta conectarse

COORDINACIÓN: LA CONDICIÓN DE CARRERA

Dos réplicas no deben tomar el mismo lote; provocamos el defecto antes de resolverlo, y ambas versiones están en el historial

Reclamo en dos pasos (versión con defecto)



ambas procesan #42: lote duplicado

Extracción atómica (comando BRPOP de Redis)



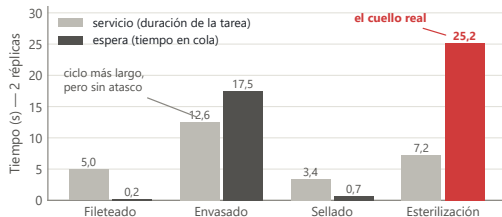
cada lote llega a exactamente una réplica

- Reclamar en dos pasos —*mirar* y después *sacar*— deja una ventana en la que otra réplica ve el mismo lote. **Prueba:** 3 réplicas, 200 órdenes → órdenes procesadas dos veces.
- BRPOP (comando de Redis) entrega el elemento y lo elimina en **una operación indivisible**; Redis atiende de a uno, así que dos réplicas no pueden recibir el mismo lote. Misma prueba: **0 duplicados** — la ventana **se elimina**.

CRITERIO DE CUELLO: ESPERA, NO SERVICIO

Un ciclo largo no implica atasco; atasco es que llega trabajo más rápido de lo que se drena — y eso se ve en la espera

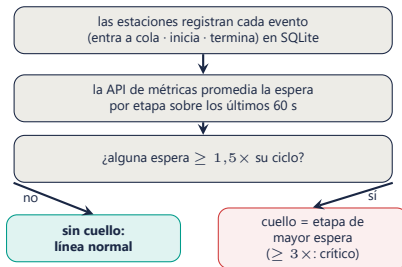
- De los eventos de cada lote se derivan dos tiempos por etapa:
 $\text{espera} = t_{\text{inicio}} - t_{\text{entra_cola}}$
 $\text{servicio} = t_{\text{termina}} - t_{\text{inicio}}$
- El *servicio* solo dice cuánto tarda la tarea; la *espera* dice cuánto creció la fila. **El cuello se localiza por espera.**
- El dato de la derecha lo demuestra: con 2 réplicas, envasado sigue teniendo el ciclo más largo (12 s), pero la fila que crece está en **esterilización** (7 s). Un criterio por servicio jamás la habría encontrado.



EL DETECTOR EN FUNCIONAMIENTO

La lógica que corre cada vez que alguien consulta el diagnóstico — y cómo la usamos para evidenciar el cuello

- **La regla:** *advertencia* si la espera promedio (ventana de 60 s) supera $1,5 \times$ el ciclo de la etapa; *crítico* si supera $3 \times$. El cuello es la etapa de **mayor espera** entre las que superan umbral; si ninguna, no hay cuello.
- **Cómo lo evidenciamos:** con la línea a ritmo constante, el detector señaló **envasado** (67,8 s). Agregamos una segunda envasadora: la espera bajó, pero la fila reapareció en **esterilización**. Con una tercera, envasado quedó sobrado — y el límite siguió ahí.



DEMO EN VIVO

El sistema completo, corriendo — tablero en localhost:8501

1. Línea al día: 4 etapas, 5 réplicas visibles con su ciclo y su carga.
2. Ingresamos lotes desde la interfaz, espaciados (~5 s).
3. La espera de esterilización crece en el gráfico: cartel **ámbar** → **rojo**, con la razón del diagnóstico.
4. **Condición de carrera en vivo**: un botón corre lado a lado las dos versiones del reclamo con 100 órdenes. La versión en dos pasos registra ~300 envasados para 100 órdenes (duplicados); la atómica, exactamente 100.
5. Detenemos Redis: la interfaz **nombra al servicio caído** y el resto sigue vivo. Lo levantamos y todo continúa.

Qué mirar durante la demo

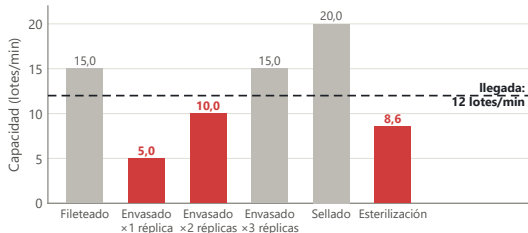
El color marca *importancia*, no identidad: lo normal es gris; el color fuerte queda reservado para advertencia y crítico, siempre con icono y texto.

1 comando

`docker compose up` — despliegue completo

EXPERIMENTOS: QUÉ CORRIMOS Y POR QUÉ

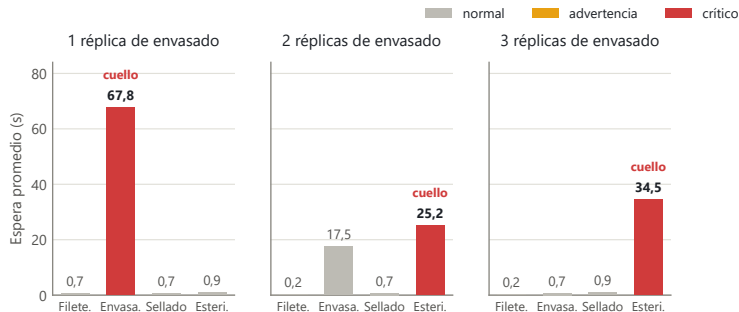
El mismo escenario, repetido con 1, 2 y 3 envasadoras — la corrida con 1 es la línea sin nuestro desarrollo



- El experimento es software: un script crea **1 orden cada 5 s durante 180 s** vía la API de órdenes y al final consulta las métricas. Ritmo constante = demanda sostenida; inyectar de golpe mide otra cosa (prueba de saturación, aparte).
- La figura anticipa el resultado: toda etapa **bajo la línea de llegada** (12 lotes/min) acumulará fila. Con 2 réplicas, envasado sube a 10 — aún insuficiente — y esterilización (8,6) queda como siguiente límite.

EL RESULTADO CENTRAL

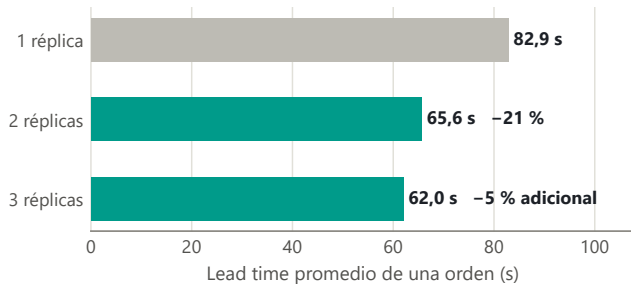
El cuello no se elimina al escalar: se muda de etapa



- «Réplicas» = contenedores de envasado en paralelo sobre la misma cola: el equivalente a instalar 1, 2 o 3 **máquinas envasadoras**.
- Con 1 (la línea sin escalar), envasado acumula **67,8 s** de espera. Con 2, el cuello **se desplaza a esterilización**. Con 3, envasado queda sobrado (0,7 s)... y el cuello sigue en esterilización.

¿CUÁNTO COMPRA CADA RÉPLICA?

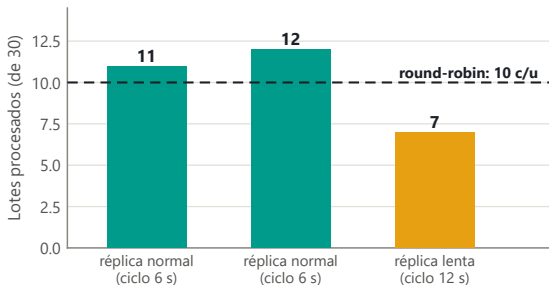
Lead time: lo que tarda una orden desde que se crea hasta que sale terminada



- Frente a la línea sin escalar, la segunda envasadora reduce el lead time un **21 %**; la tercera, casi nada: el límite ya no está en envasado.
- La siguiente inversión correcta sería **una segunda autoclave de esterilización**, no una tercera envasadora — y el tablero responde esa pregunta en vivo.

BALANCEO CON UNA RÉPLICA LENTA

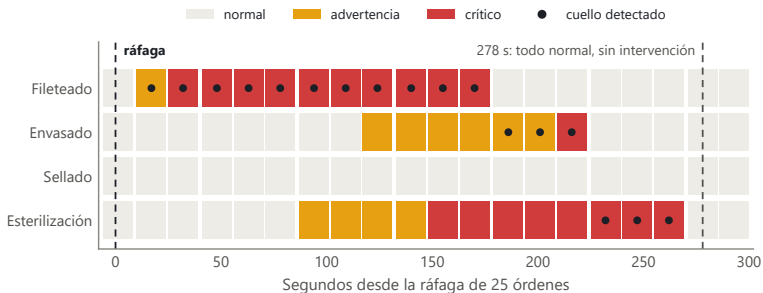
Dos réplicas normales + una al doble de ciclo, sobre la misma cola, 30 lotes



- Un reparto por turnos (round-robin) habría dado **10/10/10** sin importar la velocidad — y la fila habría crecido frente a la lenta.
- El reparto **11/12/7** no lo programó nadie: emergió de que cada réplica pide trabajo solo al quedar libre.
- Eso es balanceo *dinámico*: proporcional a la capacidad real de cada una, verificado con datos.

SATURACIÓN Y RECUPERACIÓN AUTOMÁTICA

Qué hace el sistema ante una ráfaga: 25 órdenes de golpe, sin intervención posterior



- Cada celda del gráfico es una llamada al diagnóstico de la API de métricas (la función de cuello recién vista), hecha **cada 15 s** durante la prueba.
- A los 32 s el sistema declara **crítico** solo; la congestión recorre la línea como una ola. A los **278 s** todo vuelve a normal **sin intervención**: las esperas viejas salieron de la ventana móvil.

MANEJO DE ERRORES

El error que provocamos, su causa real y la solución — medido antes y después

46 s

antes: la API quedaba muda con
Redis detenido

2,1 s

después: respuesta 503 con
mensaje que nombra al servicio
caído

Además: 201 creación · 422 validación
· 404 inexistente · 503 dependencia
caída — verificados contra el sistema
corriendo.

- **El error:** detuvimos Redis con el sistema corriendo; crear una orden dejaba a la API 46 s sin responder nada.
- **El diagnóstico:** el cuelgue no estaba en la conexión sino en la **resolución del nombre** (DNS) — al detener un contenedor, Docker elimina su nombre y la búsqueda tarda ~45 s en rendirse, fuera del alcance del timeout de conexión.
- **La solución:** un plazo duro de 2 s sobre la operación completa; si no hay respuesta, se devuelve de inmediato un 503 que **nombra al servicio que falta**, y la interfaz nunca muestra un traceback.

PERSISTENCIA Y REPRODUCIBILIDAD

Dos tablas, nada guardado dos veces; despliegue completo con un comando

- SQLite en un volumen de Docker, dos tablas: `ordenes` y `eventos` (12 eventos por orden completa).
- Espera, servicio y lead time **no se almacenan**: se calculan siempre desde los eventos — el dato guardado dos veces es el dato que se contradice.
- La base sobrevive a apagar y levantar el sistema (verificado).

La prueba de reproducibilidad: clonar el repositorio en una carpeta vacía y levantar todo — nada más que Docker instalado:

```
git clone <repo> && cd <dir>  
docker compose up -d --build  
# tablero en localhost:8501
```

8/8

contenedores *healthy* en la prueba de clon limpio

RESILIENCIA: CÓMO SE ABORDARÍA

No exigida por el enunciado — pero la arquitectura deja los puntos de corte listos

- **Estaciones:** sin estado propio, reconectan y mueren sin corromper nada. Pendiente: si una réplica muere a mitad de ciclo, su lote se pierde de la cola; el patrón estándar es moverlo a una cola «en proceso» al reclamarlo (comando BLMOVE de Redis) y devolverlo si la réplica deja de dar señales de vida.
- **Redis:** punto único de falla del reparto → una réplica de Redis con conmutación automática (Redis Sentinel), o un broker de mensajes con confirmación de entrega (p. ej. Kafka): un mensaje no confirmado vuelve a entregarse solo.
- **SQLite:** con más escritores o conmutación, migrar a MySQL/PostgreSQL cambiando solo el módulo de base de datos (bd.py) de cada servicio.
- **APIs y tablero:** sin estado; dos instancias tras un balanceador HTTP quedan en alta disponibilidad — los chequeos de salud ya existentes son su insumo.

Cada mejora toca **un** nodo sin rediseñar los demás: para eso era la división en nodos.

CONCLUSIONES

Todo lo afirmado está respaldado por mediciones reproducibles

El cuello no se elimina: se desplaza

De envasado a esterilización al pasar de 1 a 2 envasadoras; la tercera ya casi no mejora nada.

Medir la espera permite localizarlo

El cuello real tenía un ciclo más corto que envasado: por tiempo de proceso no se habría encontrado.

El balanceo por demanda se adapta solo

Reparto 11/12/7 con una réplica al doble de ciclo, sin que nadie lo asignara.

La carrera se elimina, no se administra

Extracción atómica: 0 duplicados en 200 órdenes, contra los duplicados del reclamo en dos pasos.

Las fallas se explican, no se enmascaran

Con Redis caído, la API pasó de 46 s de silencio a responder en 2,1 s nombrando al servicio ausente.

Gracias

¿Preguntas?



`github.com/CesarOlivares/Estructura-de-computadores-entrega-3`